

Part I – Classical Neural Machine Translation

1) Objective

In this section, we aim to design and implement a complete Neural Machine Translation (NMT) framework for Hindi-to-Marathi translation using recurrent neural network architectures.

2) Task Description

The task involves building a Seq2Seq translation framework consisting of an LSTM-based encoder, an LSTM-based decoder, and an Attention mechanism to improve translation quality.

3) Dataset

The dataset provided as part of the assignment was used for training and evaluation. The training dataset contains more than 2,41,000 Hindi–Marathi sentence pairs, which were utilized for training the translation models.

4) Initial Embedding

Two different types of embeddings were explored during experimentation. The first approach used randomly initialized embeddings for both Hindi and Marathi, while the second approach employed BERT-based embeddings for both languages.

5) Experimental Setup

Due to limited computational resources, the model architecture was kept relatively lightweight and trained for a smaller number of epochs. Experiments were conducted on a system equipped with an Intel Core i7 processor, 64 GB RAM, and an NVIDIA T1000 GPU with 4 GB dedicated VRAM.

6) Preprocessing and Tokenization

No additional text preprocessing was performed on the dataset.

SentencePiece tokenizers were trained separately for Hindi and Marathi and used for tokenization. Out of the total training data, the first 2,00,000 sentence pairs were used for training, while the remaining pairs were reserved for validation. The final vocabulary sizes obtained were 28,565 tokens for Hindi and 29,982 tokens for Marathi.

7) Architectural Design

We have used an encoder decoder architecture for machine translation inspired by Bahdanau et al (<https://doi.org/10.48550/arXiv.1409.0473>). They have used GRU for translation, but as per the requirement of the assignment, we have used LSTM for both encoder and decoder. Their paper mentions that GRU can be replaced with LSTM. For the coding of this section, we have taken help from their paper.

7.a) Encoder:

Embedding dimension - 128 (Random Initialization) and 768 (BERT Embedding dimension)

Hidden state dimension - $256 * 2$ (multiplied by 2 because of bi-LSTM)

Dropout - 0.3

7.b) Decoder:

Embedding dimension - 128 (Random Initialization) and 768 (BERT Embedding dimension)

Hidden state dimension - 256

Dropout - 0.2

The encoder and decoder hidden state sizes are different, the state is not directly passed from encoder to decoder, it is passed through a simple Artificial Neural Network, as a linear projection layer.

7.c) Attention

Bahdanau attention was used at the decoder side to make better use of the inputs while on the decoder side.

$$e_{t,i} = a(s_{t-1}, \mathbf{h}_i)$$

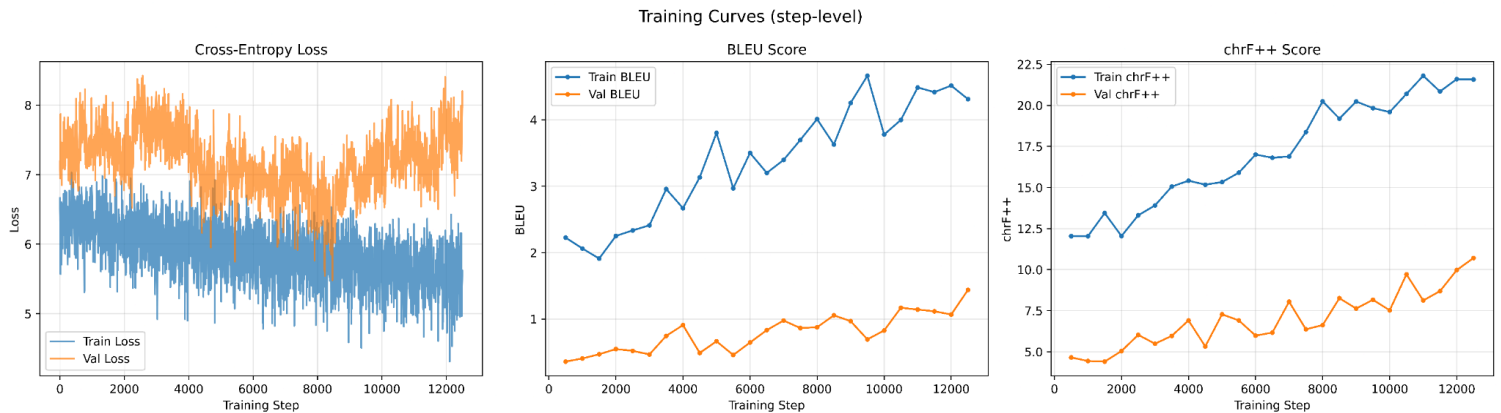
$$a(s_{t-1}, \mathbf{h}_i) = \mathbf{v}^T \tanh(\mathbf{W}[\mathbf{h}_i ; s_{t-1}])$$

$$\alpha_{t,i} = \text{softmax}(e_{t,i})$$

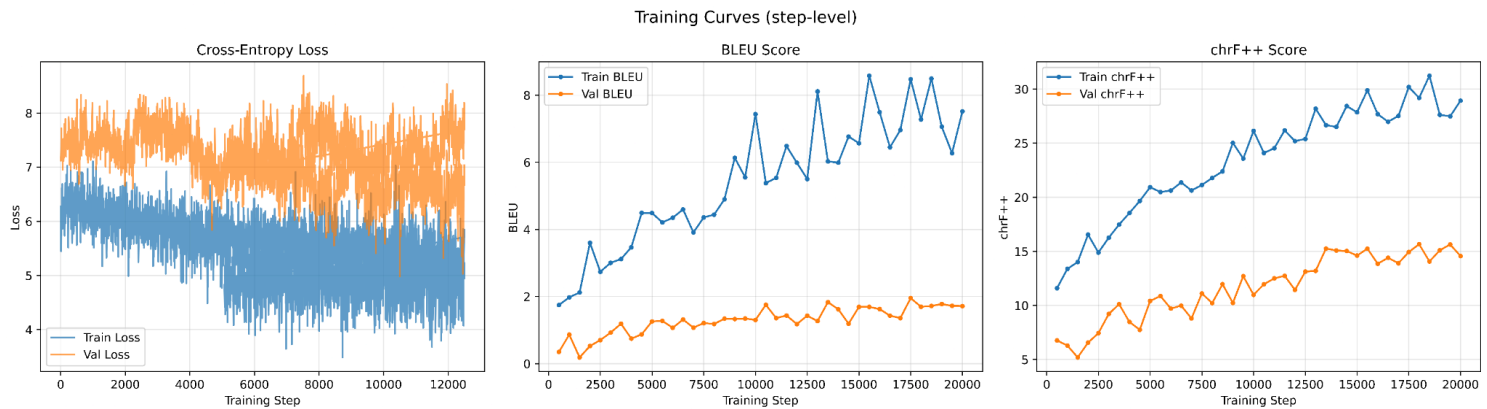
Results:

For this assignment, two different versions of the model were implemented. The first version, referred to as Experiment 1.1, used randomly initialized embeddings, while the second version, Experiment 1.2, utilized BERT-based embeddings.

To evaluate the performance of the models, Cross-Entropy Loss, BLEU-100, and CHRF++100 metrics were used. BLEU measures the similarity between the generated translation and the reference translation at the *n-gram* level, whereas CHRF++100 evaluates similarity at the character level, making it particularly useful for morphologically rich languages. The model in Experiment 1 contains 36,426,398 trainable parameters, while the model in Experiment 2 contains 95,051,038 trainable parameters. This difference primarily arises from the embedding layers. In Experiment 1, the embeddings were randomly initialized and remained trainable throughout training. In contrast, Experiment 2 used pretrained BERT-based embeddings, where the embedding layers were kept frozen during training.



Experiment 1.1 Result



Experiment 1.2 Result

The X-axis in the plots does not show epoch count, they are step count. Due to lack of computational resources and time, we have used step-wise evaluation. In this, instead of calculating loss, and scores every epoch, we calculate them after every few batches of training, these are called steps. In our experiment, step length is 5.

In both random and bert based initializations of embeddings, the losses slowly decrease, but in the bert based initialization, the rate of improvement is slightly better. The training score is normally better than the validation score, but both has similar trend of improvement. The overall score is not that well, but the plot shows a trend of improvement, so we can say that with more epochs of training and more time, the scores will improve.

The result on test data:

Experiment 1.1-

| Test BLEU: 3.96 | Test chrF++: 25.36 |

Experiment 1.2-

| Test BLEU: 8.58 | Test chrF++: 36.96 |

We can see a significant difference in the result of bert based embedding initialization. We can conclude that bert embeddings produce better results than random initialization.

Failure Analysis:

- BLEU 100 and CHRF++100 both have a range of 1-100, so our score is not even nearly good. We assume this might be because the training time is too less. With more training, this should get better.
- There is a large gap between the results of validation and testing. This might have been caused by a bad choice of validation set.
- The losses and the scores fluctuate a lot. This might have been the cause of bad choice of learning rate, and low training time. We can make better learning rate schedulers for better and smooth results.

Part II – Language Model Pretraining and Translation

BERT : BERT encoder model is trained from scratch for hindi language understanding.

Experimental Setup

As we lack computational resources, the architecture parameters have been reduced, and run for a small number of epochs. The machine has an Intel core i7 processor with 64 gb RAM. We used an Nvidia T1000 GPU with 4gb dedicated vRAM. Python 3.10 was used as the programming language.

Dataset:

The dataset used for training the model to build up the understanding of Hindi language was taken from AI4Bharat Sangraha (verified versions), a subset, for Hindi language (0.25M) from the huge corpus of Indian language data.

Data Preprocessing :

- Remove English alphabet characters.
- Remove numeric values.
- Normalize white space.
- Remove short sequences.

Tokenization :

For the tokenization, SentencePiece Tokenizer was used to convert a subword tokenization algorithm that converts raw text into smaller units (tokens) before feeding them into a Transformer. Unlike traditional tokenizers that split on spaces first, SentencePiece treats the input as a sequence of Unicode characters and learns token boundaries automatically.

text → tokens → IDs

Consider a sentence :

मेरा नाम जिश्नु है

Possible tokenization :

["_मेरा", "__ नाम", "__ जिश्नु", "__ है"]

IDs :

[89, 967, 123, 56]

The table below shows the setup, where, UNK is “unknown”, PAD is “Padding”, BOS is "Beginning of the sentence" and EOS is “Ending of the sentence”.

Configuration :

Parameters	Value
Vocabulary Size	20,0000
Model Type	BPE
Character Coverage	0.9995
PAD	0
UNK	1
BOS	2
EOS	3
CLS	4
SEP	5

MASK	6
------	---

Sequence Formatting

Each sequence becomes:

[CLS] token1 token2 ... tokenN [SEP] [PAD] [PAD]

Data Preparation :

The diagram below illustrates the data preparation pipeline used for Hindi text before it is passed into the BERT encoder. The process begins with the Hindi Text, which consists of the raw dataset containing sentences written in Hindi. Since neural networks cannot process raw text directly, the text is first converted into tokens using SentencePiece Encoding.

After tokenization, the encoded text moves to the Sequence Packing stage. In this step, tokenized sentences are organized into fixed-length sequences suitable for training the model. Rather than storing each sentence individually, multiple shorter sequences can be combined together efficiently. This helps minimize unused space and improves GPU utilization during training.

Next, the packed sequences undergo Padding. Deep learning models require inputs within a batch to have the same length, so shorter sequences are extended using a special padding token, such as [PAD], until they reach the required size. This ensures that all input tensors have consistent dimensions during training.

Once preprocessing is finished, the dataset is divided into Training and Validation sets. The training set is used to learn the model parameters, while the validation set helps evaluate performance during training and reduces the risk of overfitting.

Finally, the processed data is stored as NumPy Shards. Instead of saving the entire dataset as one large file, it is split into multiple smaller .npy files, known as shards. This approach enables faster data loading, lower memory consumption, and more efficient parallel processing during large-scale pretraining.

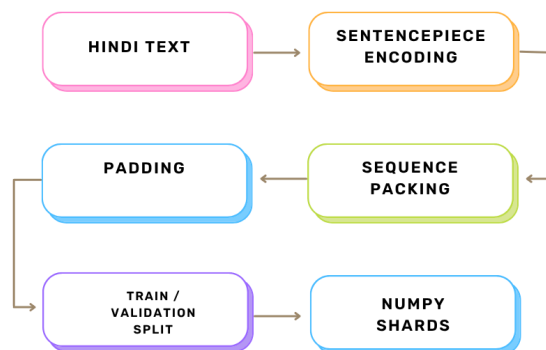
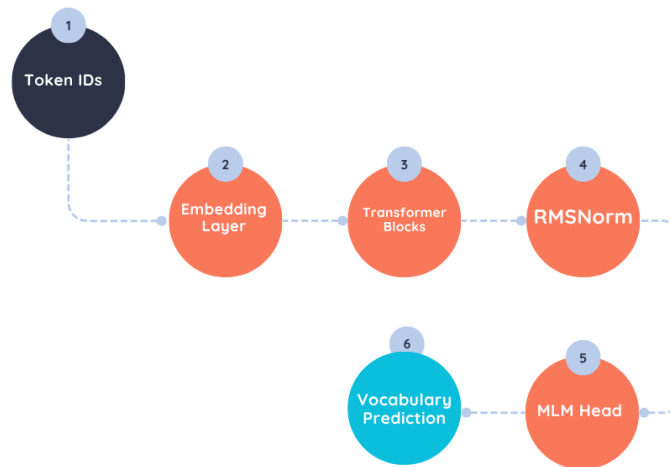


Table below shows the configurations:

Parameter	Value
Validation Split	10%
Sequence Length	128
Shard Size	10M

Model Architecture :

The diagram shows the forward pass of our Hindi BERT-style Masked Language Model (MLM) during pretraining.



1. Token IDs (Input):

This is the text after SentencePiece tokenization, later converted to IDs.

Output shape: (B,T)

(batch size × sequence length) where total_batch_size =2048

Example:(4,128)

2. Embedding Layer:

The model cannot understand integers directly. Embedding converts each token ID into a dense vector. The embeddings are randomly initialized.

For example :

145 $\rightarrow$ [0.21,-0.52,0.13,...]

87 $\rightarrow$ [0.89,0.33,-0.14,...]

The embedding size for the BERT model is: 768

then:

$(B,T) \rightarrow (B,T,768)$

Meaning:

$(4,128) \rightarrow (4,128,768)$

Now every word becomes a semantic representation.

3.Transformer Blocks :

This is the main learning component.Each Transformer block contains:

Attention $\rightarrow$ Residual Connection $\rightarrow$ Normalization $\rightarrow$ Feed Forward Network

Attention :

Positional information is essential in the attention mechanism because the Transformer architecture processes all tokens simultaneously and does not naturally capture the order of words in a sequence. Traditional Transformers address this limitation using positional embeddings. However, in our architecture, RoPE (Rotary Positional Embedding) is used instead of standard positional embeddings.

RoPE incorporates positional information by applying rotational transformations to the Query (Q) and Key (K) vectors based on the position of each token before the attention scores are computed. This allows the model to capture relative positional relationships between

tokens more effectively while preserving the efficiency of the attention mechanism.

The attention becomes:

$$Attention(Q, K, V) = softmax \left(\frac{Q_{rot} K_{rot}^T}{\sqrt{d_k}} \right) V$$

where:

$$Q_{rot} = R(m)Q, \quad K_{rot} = R(n)K$$

- $m, n \rightarrow$ positions of tokens
- $R(\cdot) \rightarrow$ rotation matrix

Group Query Attention : Instead of using standard Multi-Head Attention, the architecture employs **Grouped Query Attention (GQA)**. GQA is an optimized variant of Multi-Head Attention designed to reduce memory usage and improve inference speed while maintaining performance close to that of standard attention mechanisms.

In traditional **Multi-Head Attention (MHA)**, each attention head has its own separate **Query (Q)**, **Key (K)**, and **Value (V)** projections. Although this approach provides strong performance, it requires storing a large number of Key and Value tensors, which increases memory consumption.

Grouped Query Attention addresses this issue by keeping multiple Query heads while reducing the number of Key and Value heads. In GQA, several Query heads share the same Key and Value projections within a group. This significantly lowers memory requirements and improves computational efficiency, especially during large-scale training and

inference, while still preserving most of the representational power of standard Multi-Head Attention. Attention becomes:

$$Attention(Q_i, K_g, V_g)$$

where:

- $Q_i \rightarrow$ individual query head
- $K_g, V_g \rightarrow$ shared key/value group

The table below shows the parameters used in GQA :

Hyperparameter	Value
Layers	4
Hidden size	768
Attention Heads	4
KV heads	2
Vocabulary	20,000
Sequence Length	128

Residual Connection:Residual connection (skip connection) is added in attention layers to make deep Transformer networks easier to train and preserve information from earlier layers.

Normalization: Normalization is required to properly scale the input vectors. Here Root Mean Square Normalization or RMSNorm has been used.

Formula:

$$RMSNorm(x) = x \frac{1}{\sqrt{mean(x^2) + \epsilon}}$$

Feed Forward Network:

A simple FFN or feed forward neural network is used after the residual connection to adjust the dimension of the input, otherwise constantly adding residual connection will increase vector size. This layer aggregates information and maintains dimension.

4. Masked Language Model :

The training strategy used is **BERT-style Masked Language Modeling (MLM)** with dynamic masking. The primary objective of MLM is to train the model to understand contextual relationships in language by predicting missing words within a sentence. In this approach, the model is not given the complete input sequence directly. Instead, certain tokens are randomly masked or replaced, and the model is trained to predict the original tokens using the surrounding context. This enables the model to learn rich bidirectional representations of language. Dynamic masking further improves training by changing the masked positions during different training iterations, rather than masking the same tokens every time. As a result, the model is exposed to a wider variety of masking patterns, which improves generalization and leads to more robust language understanding.

Dynamic Masking :

Dynamic masking means mask positions are generated during training, not fixed beforehand.

Epoch 1:

राम [MASK] गया

Epoch 2:

[MASK] स्कूल गया

Epoch 3:

राम स्कूल [MASK]

This exposes the model to more prediction patterns and improves generalization.

The 80-10-10 Rule

After selecting tokens for masking, BERT does **not always replace them with [MASK]**.

80% → Replace with [MASK]

Example:

मैं खाना खाता हूँ

↓

मैं [MASK] खाता हूँ

Model predicts: खाना

This teaches missing-token prediction.

10% → Replace with random token

Example:

मैं किताब पढ़ता हूँ

↓

मैं आम पढ़ता हूँ

Target : किताब

This prevents the model from depending too much on the [MASK] symbol.

10% -> Keep unchanged

Example:

मैं किताब पढ़ता हूँ

Target: किताब

The word remains visible, but the model still tries to predict it. This helps reduce mismatch because during inference there will be no [MASK] tokens.

The Mask Ratio was kept 15% that is Only 15% of tokens in each sequence are chosen for prediction.

The table below represents the optimization specifications that was used in the architecture :

parameter	Value
Optimizer	AdamW
Max Learning Rate	1e-4
Min Learning Rate	1e-5
Scheduler	Cosine Decay
Warmup	1000
Max Steps	20,000

The model training uses the AdamW optimizer together with a Cosine Learning Rate Scheduler and Linear Warmup. These settings are selected to achieve stable optimization and efficient convergence during BERT pretraining.

This strategy consists of two phases:

1. **Linear Warmup**
2. **Cosine Decay**

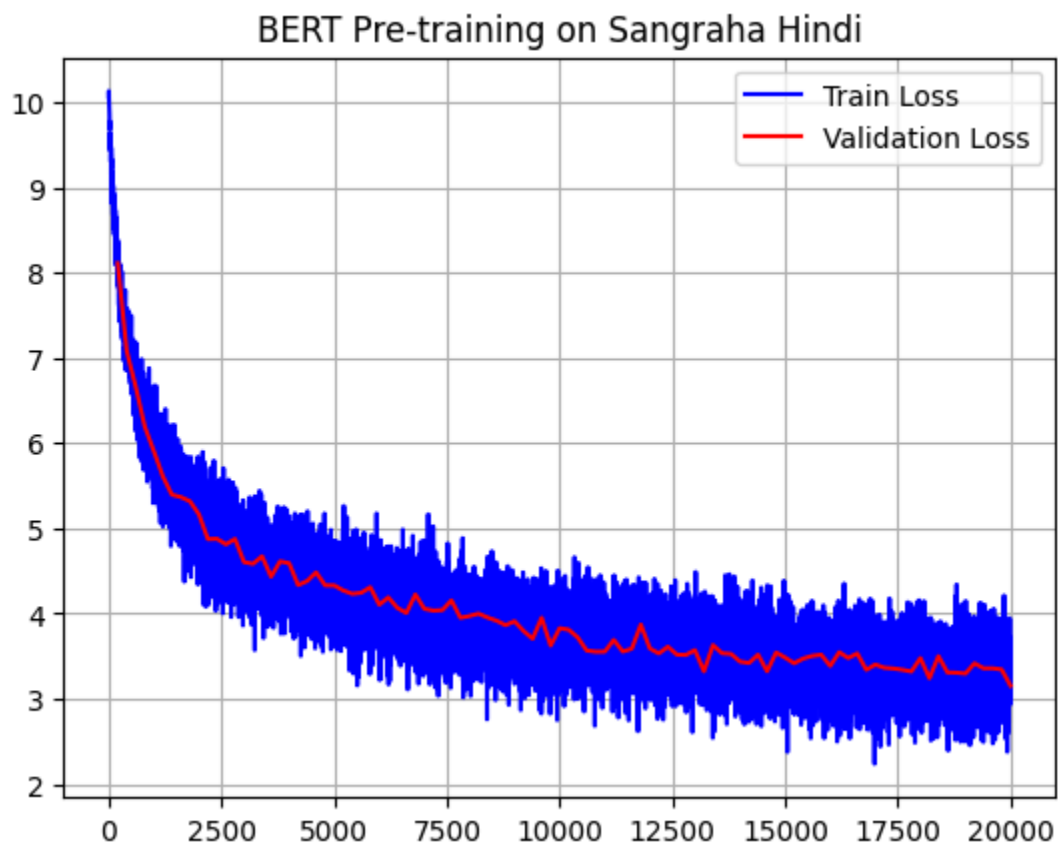
The table below shows the parameters of batching :

Parameters	Value
Global Batch	2048
Micro Batch	4
Sequence Length	128

Gradient accumulation is used.

Result:

The figure given below represents the train loss vs validation loss of BERT training from scratch :



The figure clearly shows that both the training loss and validation loss decrease steadily over time, indicating that the model is learning effectively. The curves also suggest that the model had not yet reached saturation or convergence, as training was limited to only 2 epochs due to computational constraints. With access to better computational resources and longer training duration, the model performance could likely improve further, potentially resulting in lower loss values and better overall language understanding.

GPT-2 : GPT decoder model is trained from scratch for Marathi language understanding.

DATASET :

The dataset used for training the model to build up the understanding of Hindi language was taken from AI4Bharat Sangraha (verified versions), a subset, for Marathi language (1M) from the huge corpus of Indian language data.

The experimental setup, data preparation and preprocessing, hyperparameter, GQA, RMSNorm, RoPE etc. were kept the same as the BERT model, only the difference is in the Masking.

GPT is an autoregressive model that means GPT generates text one token at a time, where each new token depends only on previously generated tokens.

This means :

$$P(\textit{sentence}) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \dots$$

The model predicts the **next token given all previous tokens**.

Example

Input:

भारत की राजधानी

GPT predicts:

Step 1:

भारत की राजधानी → दिल्ली

Step 2:

भारत की राजधानी दिल्ली → है

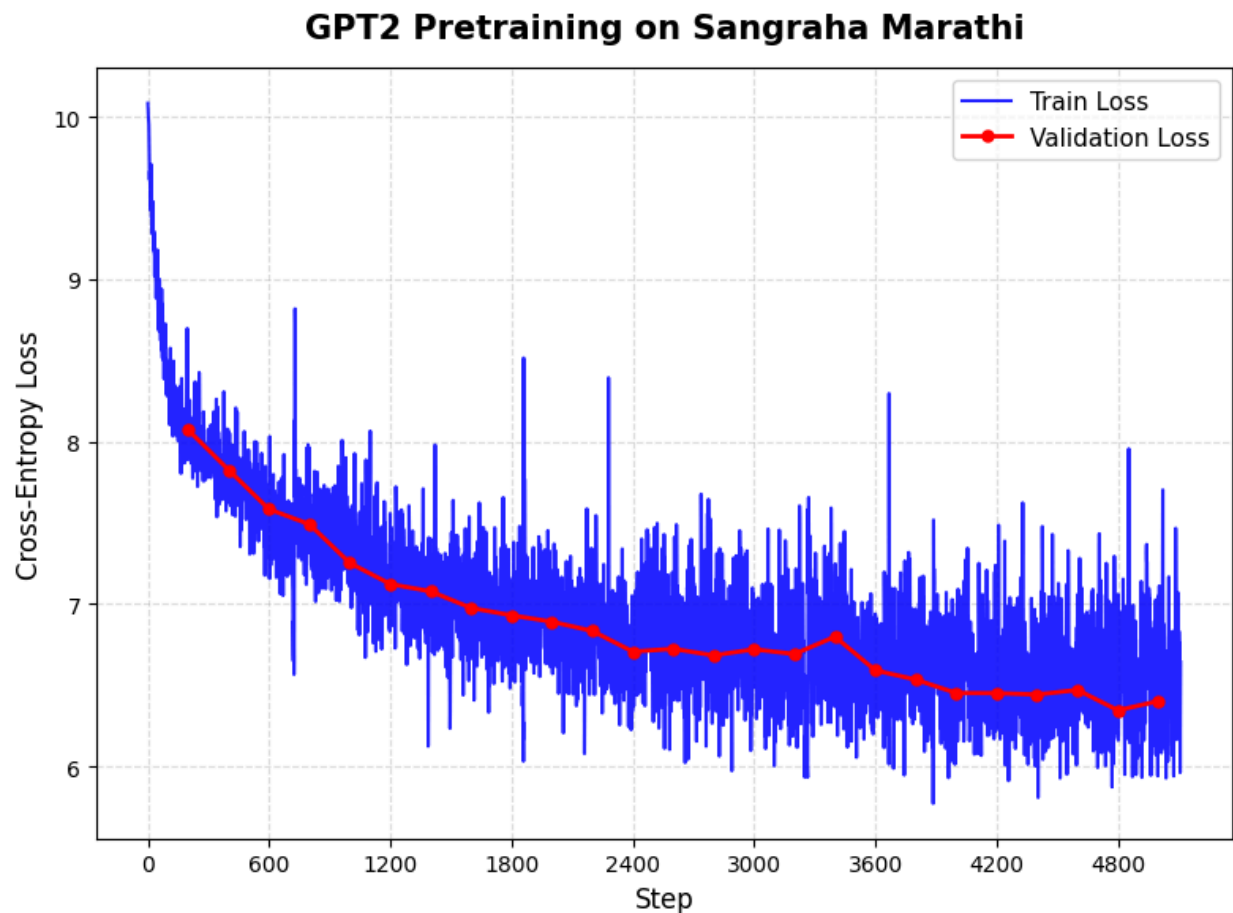
Final output:

भारत की राजधानी दिल्ली है।

⇒ GPT enforces Causal Masking .

Result :

The figure given below represents the train loss vs validation loss of BERT training from scratch :



Here also we can see that training as well as validation loss is decreasing over the period of time and didn't attend saturation yet because the

number of epochs was kept 2 due to computational constraints. If the resource would have been better the result can potentially improve a lot.

Hindi to Marathi Neural Machine Translation:

Experimental Setup:

For the final stage fine-tuning, we have used a kaggle workspace. It has 30 gb RAM, 2 T4 GPU with 15 gb vRAM each. We have used Python 3.12.12 as the programming language.

Model Training:

For Hindi to Marathi translation , BERT model and GPT 2 model pretrained from scratch using AI4Bharat Sangraha (verified versions) data is used. Then, they are combined for translation using cross-attention and after that fine tuned on the data provided.

Cross-Attention : Cross-attention is the mechanism that allows the decoder to look at the encoder output while generating translations.

In this model:

- The encoder processes the Hindi sentence.
- The decoder generates the Marathi sentence.
- Cross-attention acts as the bridge between them.

The idea is:

- Query (Q) comes from decoder and tells What information is needed now
- Key (K) and Value (V) come from encoder outputs where Key(K) represents source words and Value (V) represents actual information passed to the decoder.

The attention scores are computed using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

This computes similarity between:

- the current decoder token representation,
- and all encoder token representations.

The decoder then focuses more strongly on relevant Hindi words while generating each Marathi word.

For example:

Hindi input:

मैं स्कूल जाता हूँ

While generating Marathi word:

शाळेत

the decoder attends strongly to the Hindi word:

स्कूल

Thus , cross-attention performs **soft alignment** between source and target languages.

Padding Mask in Cross-Attention :

The model also masks padding tokens.This ensures that:

- PAD tokens in the Hindi sentence receive near-zero attention,
- Only real words contribute to translation.

The role of Cross-Attention in this architecture is that Cross-attention is the newly added component that connects both models.

⇒The weights of the Cross-attention are new and randomly initialized.

Teacher Forcing :

Teacher forcing is a training technique used in sequence-to-sequence models where the decoder is fed the actual target token from the dataset instead of its own previously generated prediction.

In this implementation, teacher forcing is created during dataset preparation itself.

Decoder input : BOS + tokens

Decoder targets: tokens + EOS

Suppose the Marathi sentence is:

मी शाळेत जातो

After tokenization:

[मी, शाळेत, जातो]

Then:

Decoder Input : [BOS, मी, शाळेत, जातो]

Target Output : [मी, शाळेत, जातो, EOS]

So during training:

- At time step 1, decoder receives BOS and predicts मी
- At time step 2, decoder receives the **correct token** मी and predicts शाळेत
- At time step 3, decoder receives शाळेत and predicts जातो

Thus, the decoder always sees the correct previous token during training. Without teacher forcing, the decoder would feed its own predicted token back as the next input, causing errors to accumulate rapidly during

training. The decoder input is therefore called a shifted-right target sequence.

Beam Search Decoding :

Beam search is a decoding technique used during inference, particularly in machine translation, to generate higher-quality translations compared to greedy decoding. In greedy decoding, the model selects only the most probable token at each step, which can sometimes lead to suboptimal translations because earlier choices cannot be revised. In contrast, beam search maintains multiple possible candidate sequences simultaneously, allowing the model to explore different translation paths before selecting the final output.

At every decoding step, beam search retains the top k most probable candidate sequences, where k is known as the beam width. By considering several possible continuations instead of just one, beam search increases the likelihood of generating more accurate and fluent translations.

Here:

- k = beam width (4 in our code)
- multiple hypotheses are maintained simultaneously.

For beam width = 3:

Candidate	Probabillity
मी शाळेत	0.78
मी घरी	0.56

आम्ही शाळेत	0.30
-------------	------

At the next step, all candidates are expanded again and rescored. This allows the model to recover from locally suboptimal choices and generate more fluent translations. Each beam stores, token sequence which is partially generated sentence and score which is cumulative log probability.

At every decoding step:

1. Each candidate sequence is fed into the decoder.
2. The model predicts probabilities for the next token.
3. Top-k next tokens are selected.
4. New candidate sequences are formed.

Model Training :

Two-Phase Training Strategy is used :

Phase 1: Decoder + Cross-Attention only (encoder frozen): The encoder produces stable Hindi representations; only the decoder and the newly-initialized cross-attention weights are updated. This is fast and avoids catastrophic forgetting.

Phase 2: Joint fine-tuning (encoder unfrozen): Use a much lower learning rate for the encoder ($10\times$ smaller) so it adapts toward translation without losing its Hindi language understanding.

⇒ Number of epochs is kept 2 due to computational constraints.

Results:

BLEU 100 (left): focuses on exact n-gram overlap with reference translations.

ChrF++100 (right): character-level metric, usually smoother and more sensitive to morphology/partial matches.

BLEU Score Analysis:

BLEU fluctuates between roughly:

- Train: 4.0–8.1
- Test: 3.0–7.9

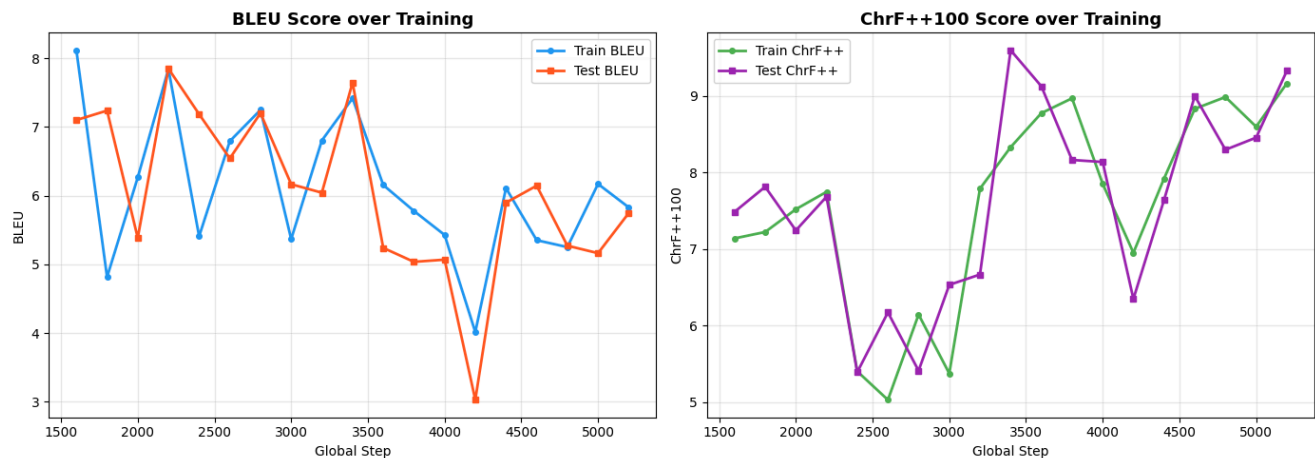
Peaks occur around:

- step ~2200
- step ~3400

Significant drop around:

- step ~4200

The training and validation curves overlap significantly, indicating that the model is not excessively memorizing the training data. This is likely because the model was trained for only 2 epochs and the number of parameters was kept relatively limited due to computational constraints. As a result, the improvements learned during training are also reflected on unseen validation data. Additionally, the close alignment between the training and validation curves demonstrates good generalization capability, suggesting that the model is able to perform consistently on both seen and unseen data without significant overfitting.



ChrF++100 Analysis :

- Scores improve overall: from ~7.1 -> ~9.2
- Test and train remain close.
- Less noisy than BLEU.

Unlike BLEU, ChrF++ shows:

- more consistent learning
- gradual quality improvement

This suggests:

- translations are improving at the character/subword level
- the model is learning morphology and partial word correctness

Highest ChrF++ occurs near:

- step ~5200

Failure Analysis :

- In BLEU100 plot the curves overlap a lot, showing the model is not memorizing training data excessively due to the number of epochs being 2 as well as the number are parameter less due to computational constraint.

- Test BLEU falls to about 3.0, Since it recovers afterward, it's probably not permanent divergence.
- In CHRF++100, Test scores sometimes exceed train scores This can happen because the test set may be easier or sampling variance .